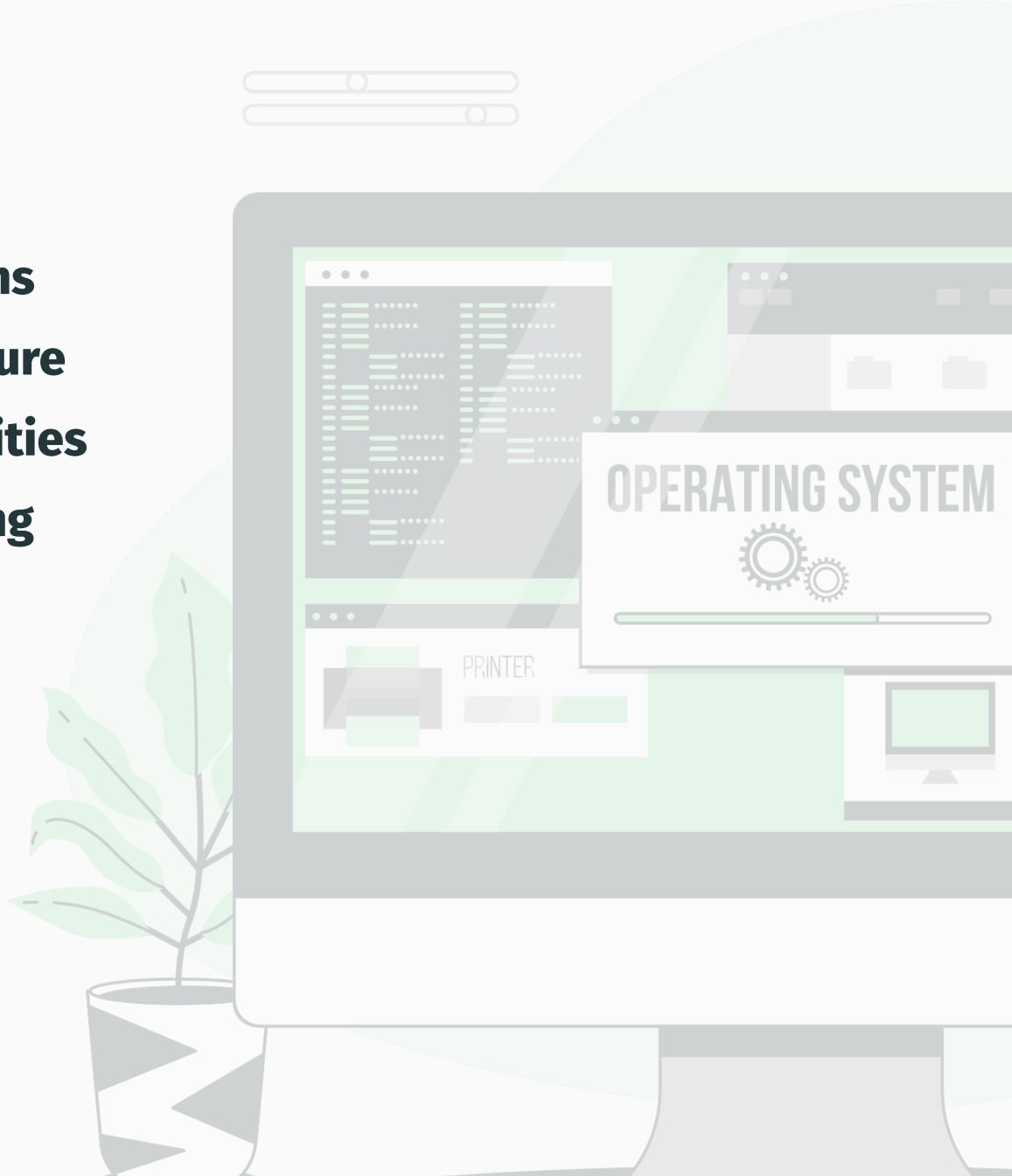
MATH 177
Computing for Mathematics

3 credit hours

Dr. Isabel Mensah
Department of Mathematics

Lecture 4: OPERATING SYSTEMS

- Introduction
- History of Operating Systems
- Operating System Architecture
- Coordinating Machine Activities
- Handling Competition Among Processes
- Security



Introduction

Introduction

Introduction

- **An Operating System** is a program which controls the execution of all other programs (applications)

Introduction

- **An Operating System** is a program which controls the execution of all other programs (applications)
- It acts as an **intermediary** between the user(s) and the computer.

Examples of OS

Windows

FreeBSD

Mac OS

redhat

solaris

Sun Cobalt

Tasks of the operating System

Recognises and installs peripheral devices

Manages files and folders

Handles inputs and outputs

Shares out system memory

Loads and runs other software applications

Handles system errors and alerts user

Moves data to and from the hard disk

Allows software to communicate with hardware

Manages system security

Introduction Cont'd

- The Operating System (OS):
  - Controls all execution of multiple applications
  - Multiplexes resources between applications
    - Ensuring efficient sharing of CPU, memory, and I/O devices
  - Abstracts away from complexity
    - provides a simplified interface for users and applications to interact with hardware.

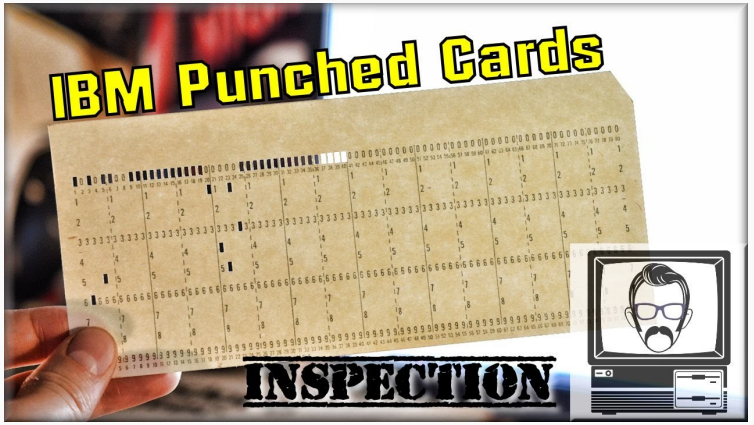
History of Operating Systems

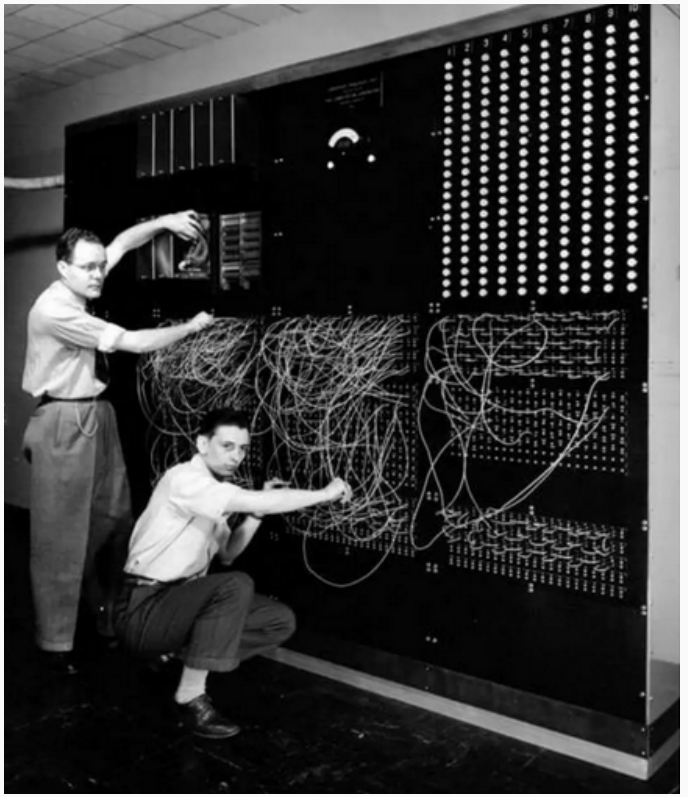
History of Operating Systems (1st generation; 1940s-1950s)

- Early computers were **large and complex**, requiring **manual setup**
- All programming was done in **absolute machine language**, often by wiring up plugboards to control the machine's basic functions
- During this generation, computers were generally used to solve **simple math calculations**
- They did not have an operating system as we know today
- Instead, they relied on manual operations to set up programs

- Running a **job** (program execution) involved:
  - **Mounting (magnetic) tapes** - stored programs and data. Operators had to load the right tape before running a job
  - **Inserting punched cards** - contained programs using holes for binary data (0s and 1s). Users had to feed cards into the computer in order to input their program.
  - **Setting switches** - manual switches (binary toggles) to enter machine instructions before execution

- Users **reserved machine time** using **sign-up sheets**
- **Slow job transitions** led to the introduction of **batch processing**



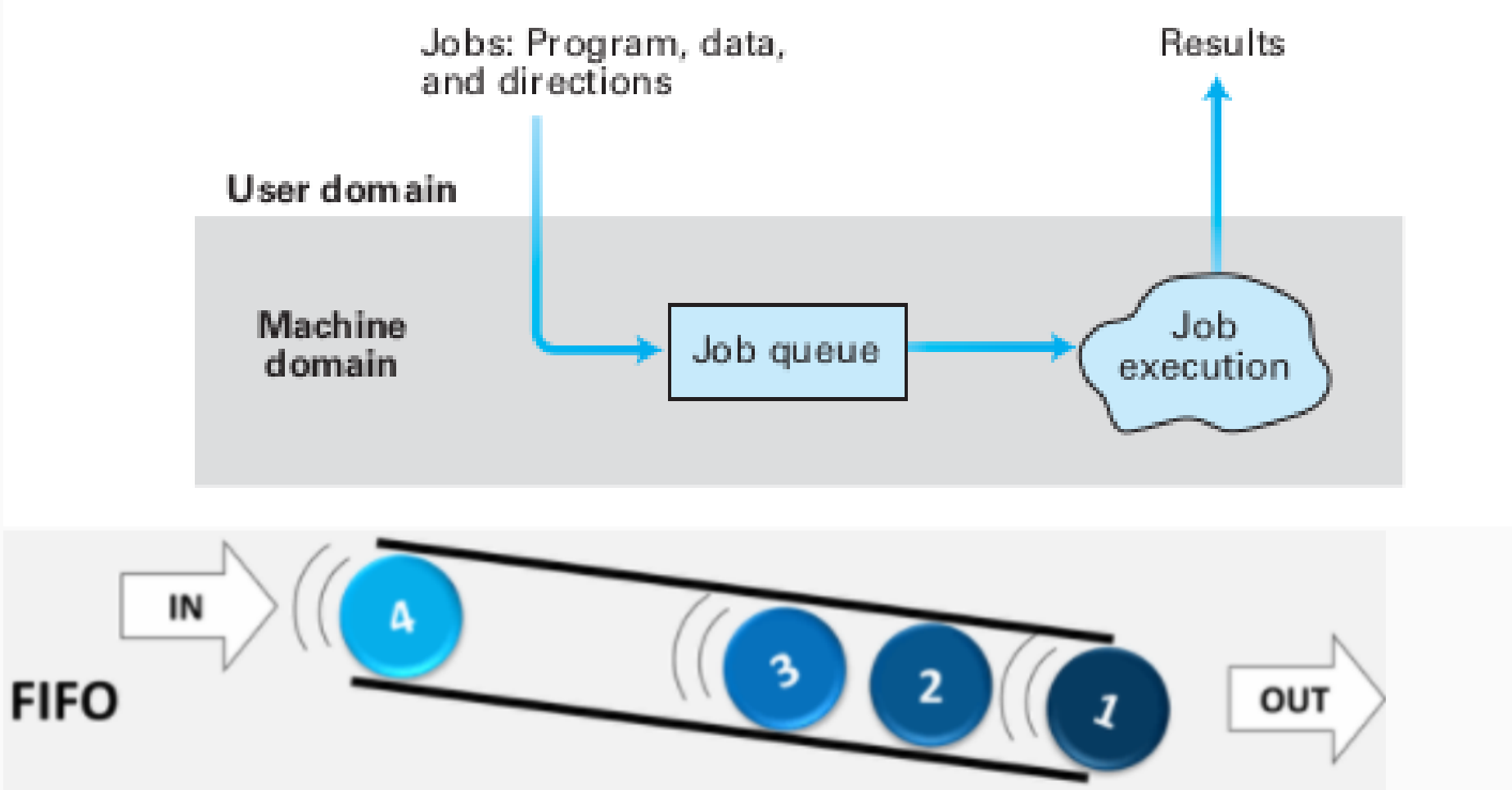


Second Generation (1955-1965) of Operating Systems

- The first operating system, GMOS, was introduced in the early 1950s by General Motors for IBM's machine, the 701
- Operating systems in the 1950s were called single-stream batch processing systems because the data was submitted in groups
- These new machines were called mainframes and were operated by professionals in large computer rooms
- Due to their high cost, only government agencies or large corporations could afford these machines.

Second Generation - Batch Processing

- Jobs were stored in a **job queue** and executed **without user interaction**
- Used a **First-In, First-Out (FIFO)** system but allowed **priority overrides**
- Introduced **Job Control Language (JCL)** for **automating setup**
- A **computer operator** handled execution, **separating users from machines**
- **Drawback:** No user interaction after submission.


Jobs: Program, data,
and directions
Results
User domain
Machine
domain
Job queue
Job
execution
IN
4
3
2
1
OUT
FIFO

IBM 701 Mainframe (1950s)

Third Generation (1965-1980) of Operating Systems

- By the late 1960s, operating system designers developed the system of multiprogramming, enabling a computer to perform multiple jobs simultaneously
- This advancement enabled the CPU to remain busy nearly 100 percent of the time, minimizing idle periods.
- The rise of real-time interaction via remote terminals (early keyboards & printers) revolutionized computing.
- Operating systems now supported word processing, reservation systems, and gaming, requiring fast response times
- Real-time processing became essential for tasks with strict deadlines.
    - **Example:** Payroll processing could be scheduled in batches, but typing in a word processor needed instant feedback.
- A major milestone was the growth of minicomputers, making computing more accessible and cost-effective.

History of Operating System: Fourth Generation - Present

- The fourth generation of operating systems saw the creation of **personal computers**, which were much more affordable than minicomputers
- A major factor in the rise of personal computing was the birth of Microsoft and the Windows operating system
- The Windows Operating System was created in **1975** when **Paul Allen** and **Bill Gates** envisioned taking personal computing to the next level
- They introduced **MS-DOS** in **1981**, which, while effective, was challenging for users due to its **cryptic commands**

MS-DOS Command Interface

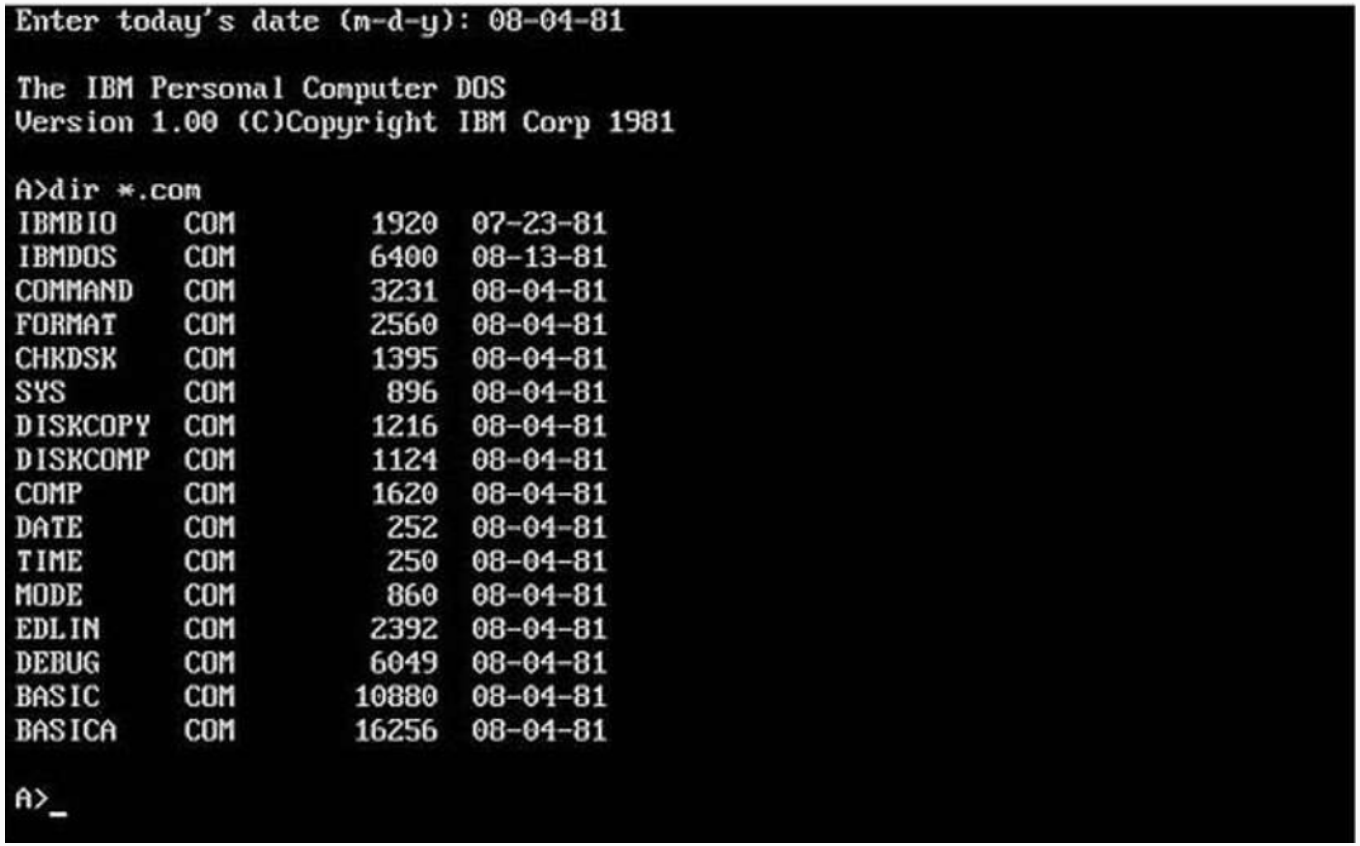


Figure: MS-DOS Command Interface (1981)

Transition to Graphical User Interfaces (GUI)

- The challenge of MS-DOS led to the development of Windows OS in the mid-1980s
- It introduced a graphical user interface (GUI) with icons, windows, and a mouse-driven interface
- Huge leap in usability

Evolution of Windows and Apple

- Microsoft released a series of operating systems, including: **Windows 95, Windows 98, Windows 2000, Windows Me, Windows XP, Windows XP Professional, Windows Vista, Windows 7, Windows 8, Windows 8.1, and Windows 10**
- Alongside Microsoft, **Apple** emerged as a major competitor in the 1980s
- **Steve Jobs**, co-founder of Apple, introduced the **Apple Macintosh**, which became a huge success due to its **user-friendly** interface
- Over the years, Windows development was influenced by the **Macintosh**, fostering a competitive dynamic between the two companies.

Modern Use of Operating Systems

- Today, our **electronic devices** run on operating systems, including:
  - **Computers** and **smartphones** - Windows, macOS, Linux, Android, iOS
  - **ATM machines** - Secure transaction processing with custom OS versions eg. Windows 10 IoT
  - **Motor vehicles** - Advanced driver-assistance systems (ADAS) & infotainment (Android Auto, Apple CarPlay)
- As **technology advances**, so do operating systems, evolving to meet new challenges and capabilities
  - AI-powered assistants (Siri, Google Assistant, Cortana)
  - Cloud computing for seamless access to apps & data
  - IoT integration, enabling smart homes & industries.

Operating System Architecture

Operating System (OS) Architecture

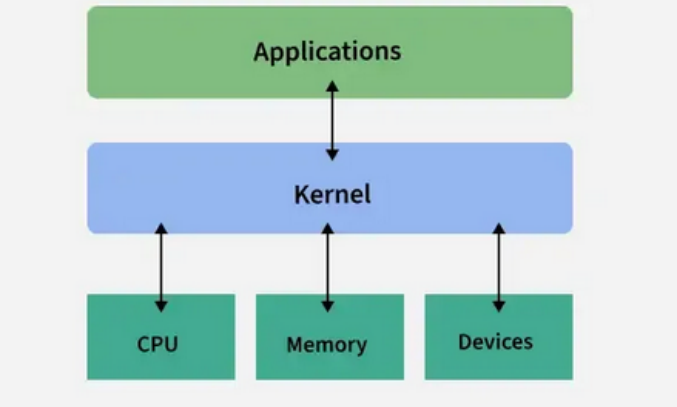
- The structure or design of an OS that dictates how its components interact and how it manages hardware and software resources

Operating System (OS) Architecture

- The structure or design of an OS that dictates how its components interact and how it manages hardware and software resources
- Major Components:
  - Kernel: The core part that manages resources (CPU, memory, I/O) and provides services via system calls

Operating System (OS) Architecture

- The structure or design of an OS that dictates how its components interact and how it manages hardware and software resources
- Major Components:
  - Kernel: The core part that manages resources (CPU, memory, I/O) and provides services via system calls



  - User Interface (Shell): Offers command-line or graphical interfaces for user interaction.

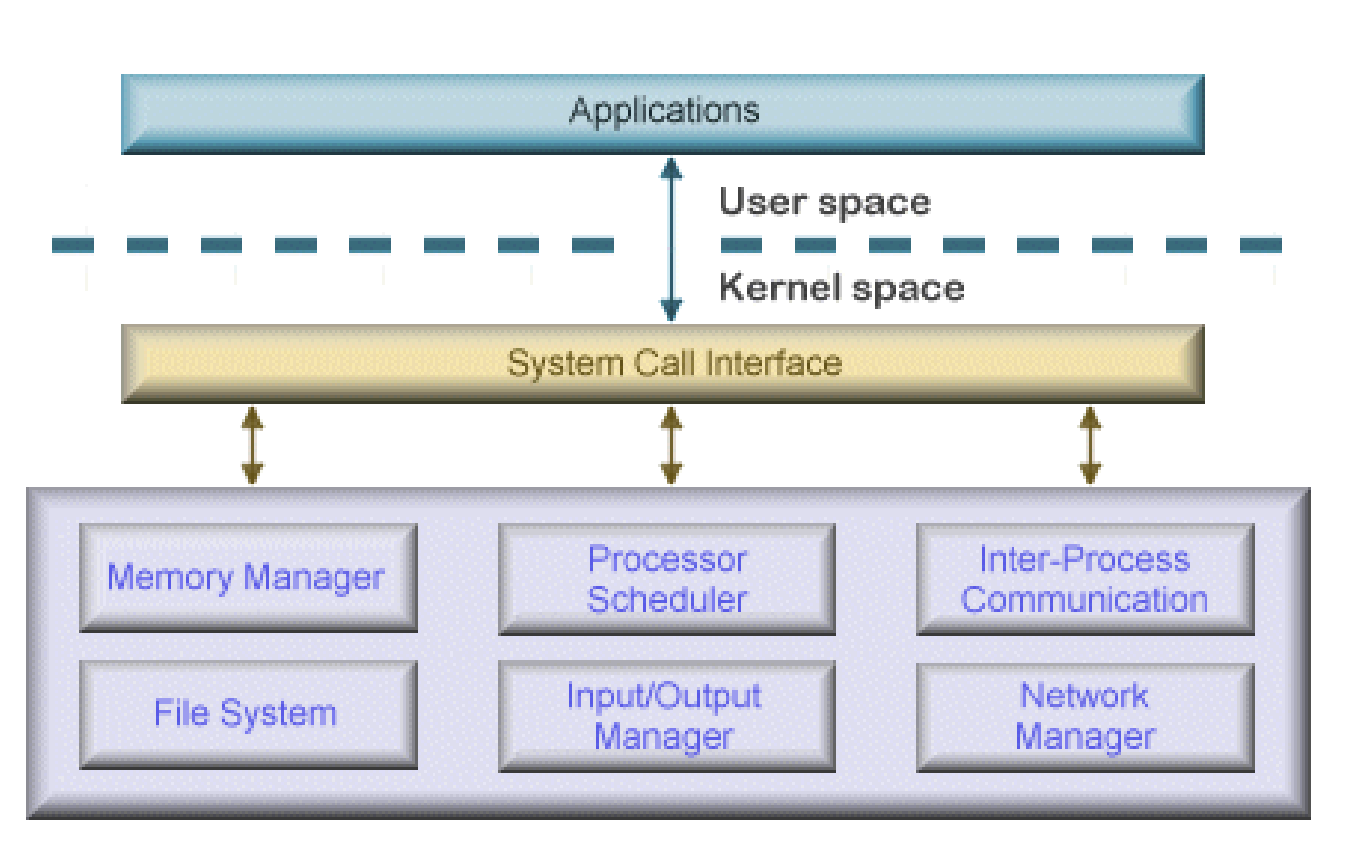
Operating System Architecture

- The kernel has **unrestricted access** to all of the resources on the system
- There are **three main types** of Operating System (OS) architectures
  - **Monolithic OS Architecture**
  - **Layered OS Architecture**
  - **Microkernel OS Architecture**

Monolithic OS Architecture

- A design where all the core services of the operating system with hardware are built into the kernel
- Core services such as managing memory, handling processes, dealing with files, and communicating
- Each component of the operating system was contained within the **kernel**
- Components could communicate directly with any other component and had **unrestricted system access**
- Advantages:
  - Made the operating system **very efficient**.
- Disadvantages:
  - **Errors were difficult to isolate**.
  - High risk of **damage** due to erroneous or malicious code.

- **A Monolithic OS Architecture**

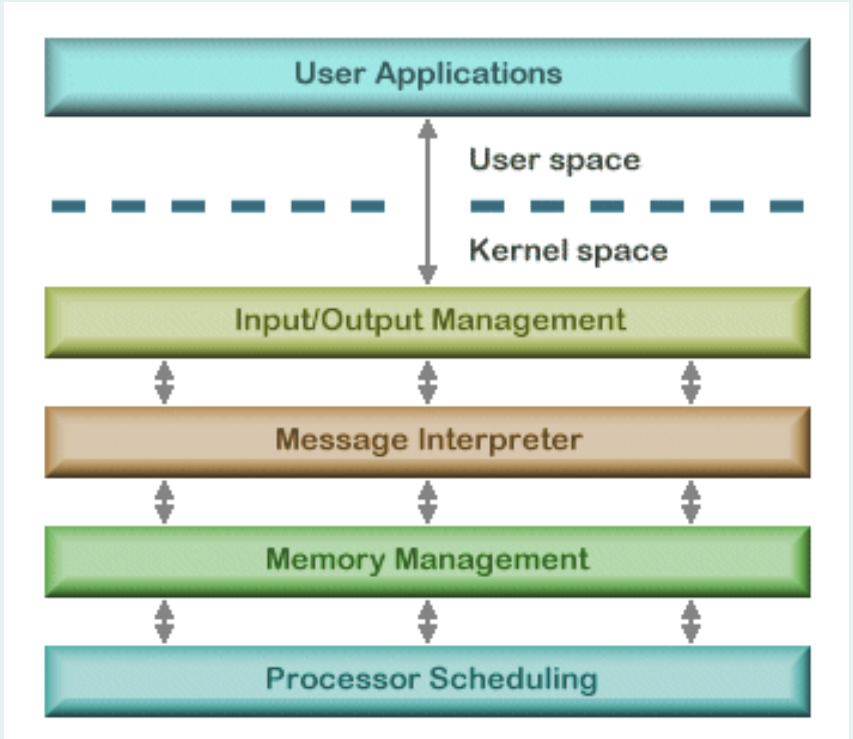


Layered OS Architecture

- An OS that's structured into layers, with each layer performing a specific function
- Each layer communicates only with the layers **immediately above and below it**
- Lower-level layers provide services to higher-level ones using an **interface** that hides their implementation
- **Advantages of Modularity:**
  - The implementation of each layer can be **modified** without requiring changes to adjacent layers.

- **Disadvantage**
- Although this modular approach imposes **structure and consistency** on the operating system, simplifying **debugging** and **modification**:
    - A service request from a user process may pass through **many layers of system software** before it is serviced
    - This results in **performance issues** compared to a monolithic kernel.

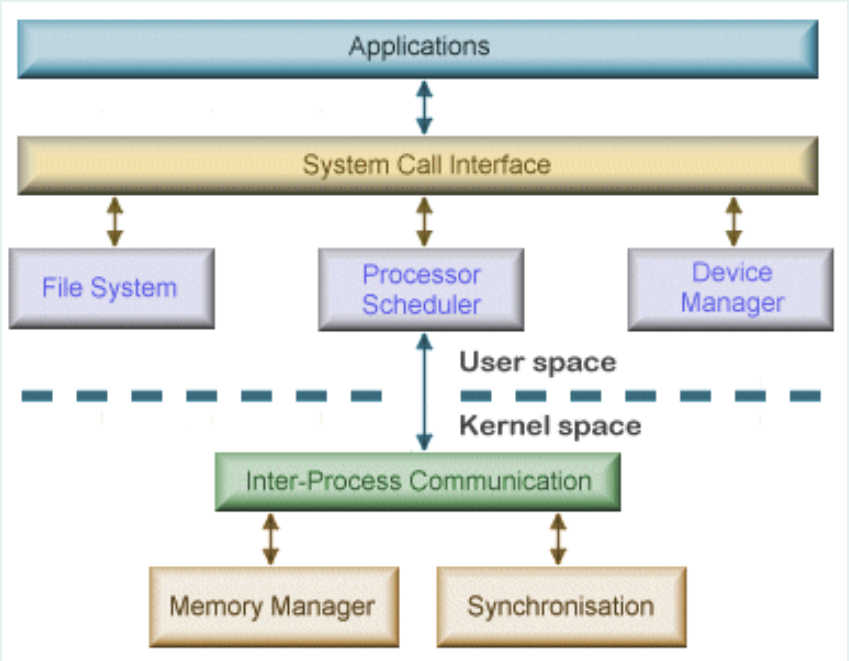
- **A Layered OS Architecture**



Microkernel OS Architecture

- The kernel includes only a **very small number of services** to keep it **small and scalable**.
- Typical services include:
  - **Low-level memory management**.
  - **Inter-process communication**.
  - **Basic process synchronization** to enable processes to cooperate.
- Operating system components outside the kernel can **fail without causing the operating system** to crash.
- **Downside:** Increased level of **inter-module communication**, which can degrade system performance.

- **A Microkernel OS Architecture**



Assignment 1.0 (To be submitted)

Briefly describe the OS architecture for Windows, MacOS and Linux
Assignment must be typed

Coordinating the Machine's Activities and Handling Competition among Processes

Coordinating the Machine's Activities

- An important task of an operating system is the **allocation of the machine's resources** to competing processes in the system

Coordinating the Machine's Activities

- An important task of an operating system is the **allocation of the machine's resources** to competing processes in the system
- Machine Resources include:
  - CPU
  - Memory
  - Storage (Disk space)
  - Input/output devices
  - Other peripheral devices

Coordinating the Machine's Activities

- An important task of an operating system is the **allocation of the machine's resources** to competing processes in the system
- Machine Resources include:
  - CPU
  - Memory
  - Storage (Disk space)
  - Input/output devices
  - Other peripheral devices
- Key components involved in resource allocation:
  - **File Manager** - Allocates access to files and mass storage space for creating new files
  - **Memory Manager** - Allocates memory space
  - **Scheduler** - Allocates space in the process table.
  - **Dispatcher** - Allocates time slices.

Time-Sharing Example

Consider a time-sharing operating system controlling a computer with a single printer. If a process needs to print its results:

- It must request access to the printer's device driver from the operating system
- The operating system must decide whether to grant the request, based on the printer's availability.

Scenario

- If the printer is **not in use**, the operating system grants the request and allows the process to continue
- If the printer is **already in use**:
  - The operating system should **deny the request**
  - The process may be classified as a **waiting process** until the printer becomes available.
- **Why is this important?**
  - Allowing two processes simultaneous access to the printer would render the results **worthless** to both.

Handling Competition among Processes

Semaphore

- A **semaphore** is used to control access to a **common resource** by multiple processes
- It helps avoid **critical section problems** in concurrent systems, such as multitasking operating systems

Cont'd

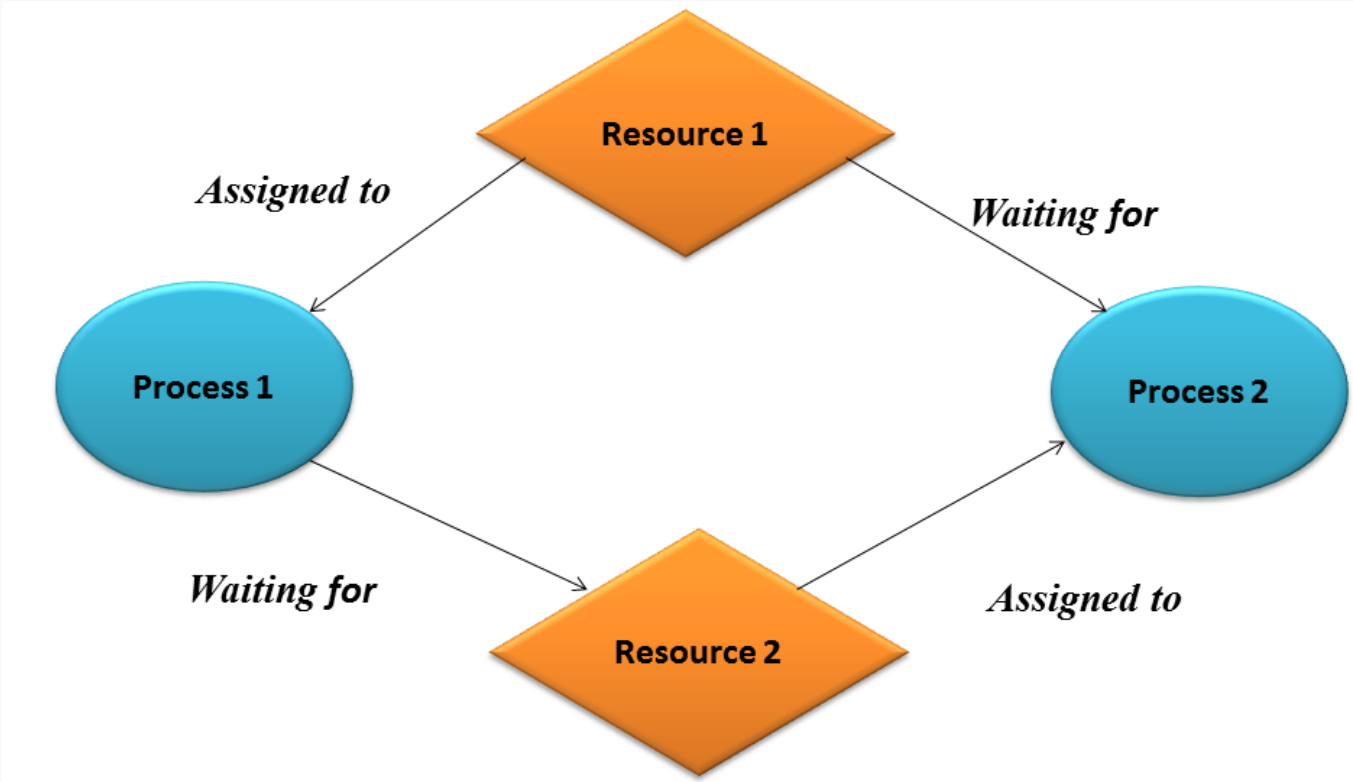
- **Critical Region:**
  - It is a sequence of instructions that should be executed by **only one process at a time**.
- **Mutual Exclusion:**
  - It is the requirement that only **one process at a time** is allowed to execute within a critical region.

Cont'd

- **Deadlock:**
  - A condition where two or more processes are **blocked from progressing** because each is waiting for a resource allocated to another process.
- **Example:**
  - One process has access to the computer's **printer** but is waiting for the **CD player**.
  - Another process has access to the **CD player** but is waiting for the **printer**.
- Such conditions can **severely degrade system performance**.

Cont'd

- **Avoiding Deadlock:**
  - Require that each process **requests all its resources at one time**.



Assignment 1.1 (No submission required)

- Reading Assignment on Security issues of the operating System

Chapter 3 section 3.5 of the book; *Brookshear, J. G. (1996). Computer Science an overview. Addison-Wesley Longman Publishing Co., Inc..*

Next Lecture

Fundamentals of Programming I

Self Improvement Tip III

Avoid expensive mistakes